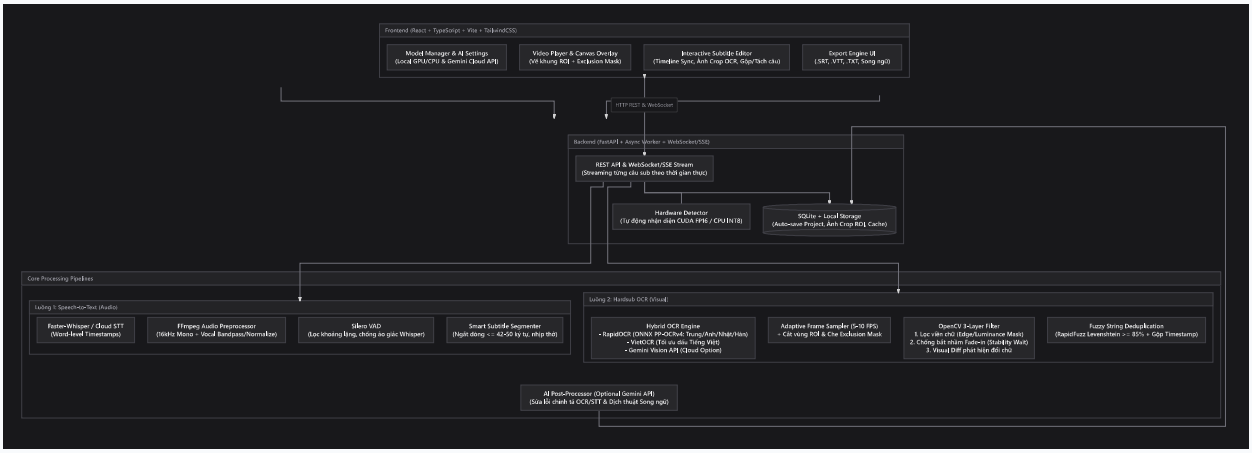
Kiến trúc Hệ thống & Kế hoạch Triển khai: Sub Magic (STT & Hardsub OCR)

Tài liệu này tổng hợp toàn bộ các quyết định kiến trúc, luồng thuật toán, cách xử lý trường hợp biên (edge cases) và lộ trình xây dựng ứng dụng **Sub Magic** dựa trên phiên thảo luận thiết kế hệ thống.

1. Tổng quan Kiến trúc Hệ thống (System Architecture)

Ứng dụng được thiết kế theo mô hình **Local Desktop Web App (Client–Server nội bộ)**, kết hợp sức mạnh của hệ sinh thái AI/Computer Vision trên Python và giao diện tương tác thời gian thực của React/TypeScript.



2. Giải pháp Kỹ thuật & Xử lý Trường hợp Biên (Edge Cases)

2.1. Luồng 1: Trích xuất từ Âm thanh (Speech-to-Text)

1. **Tiền xử lý âm thanh nhẹ và sạch (`FFmpeg`)**:
    - Trích xuất luồng âm thanh từ video sang WAV `16kHz, mono, pcm_s16le`.
    - Áp dụng bộ lọc tần số giọng nói (`highpass=f=80, lowpass=f=8000, dynaudnorm`) để giảm ù nền và cân bằng âm lượng trước khi đưa vào mô hình.
2. **Chống "ảo giác" (Hallucination) ở đoạn nhạc nền dạo đầu/cuối**:

- Bật `vad_filter=True` (Silero VAD) kết hợp kiểm tra `no_speech_threshold` và `compression_ratio_threshold` để chặn hiện tượng Whisper tự bịa chữ hoặc lặp câu vô nghĩa khi chỉ có nhạc nền.

3. **Ngắt dòng phụ đề thông minh (Smart Segmentation via Word-level Timestamps)**:
   - Bật `word_timestamps=True` trong `faster-whisper`.
   - Thay vì dùng nguyên khối segment dài của Whisper, bộ **Smart Segmenter** sẽ duyệt qua từng từ (`word`) và tự động ngắt câu khi:
     - Gặp dấu câu kết thúc (`.`, `?`, `!`, `,`) hoặc khoảng nghỉ giữa 2 từ `> 0.4s`.
     - Chiều dài dòng chạm ngưỡng cấu hình (mặc định `42-50 ký tự/dòng`, tối đa `1-2 dòng/sub`).

2.2. Luồng 2: Trích xuất Phụ đề cứng (Hardsub OCR)

Đây là bài toán khó nhất về cả hiệu năng lẫn độ chính xác. Hệ thống giải quyết qua **4 bước lọc liên hoàn**:

1. **Chọn vùng ROI + Vùng loại trừ (Exclusion Mask)**:
   - Giao diện cung cấp khung kéo-thả trực quan trên Video Player kèm Preset nhanh (`16:9 Bottom`, `9:16 TikTok Center-Bottom`).
   - Hỗ trợ vẽ các hộp **Exclusion Mask** (ô che màu đỏ) để tô đen các vùng chứa Logo/Watermark cố định nằm đè vào vùng phụ đề trước khi xử lý ảnh.

2. **Thuật toán Quét khung hình thông minh & Lọc nền chuyển động (3-Layer OpenCV Filter)**:
   - **Lấy mẫu (Adaptive Sampling)**: Đọc video ở tần suất mặc định `6-8 FPS` (tức mỗi `~125ms - 166ms` một frame, đảm bảo độ chính xác timestamp chuẩn phụ đề mà chỉ cần đọc ~1/4 số frame gốc). Chỉ giải mã (decode) và cắt đúng vùng `ROI` để tiết kiệm RAM/CPU tối đa.
   - **Lớp 1 - Tách viền chữ độ tương phản cao (Edge/Stroke & Luminance Masking)**: Cảnh phim phía sau chữ luôn chuyển động. Nếu so sánh ảnh màu RGB gốc của vùng ROI, mọi frame đều khác nhau. Do đó, hệ thống chuyển ảnh ROI sang Grayscale, kết hợp bộ lọc độ sáng cao (Luminance Threshold / Top-Hat Morphology) và phát hiện cạnh viền chữ (Sobel/Canny Edge trên vùng có viền đen bao quanh chữ sáng) để tạo ra **Text Binary Mask** (chỉ giữ lại hình dáng nét chữ, loại bỏ 90% cảnh nền chuyển động).
   - **Lớp 2 - Chờ ổn định (Stability Check - Chống Fade-in/Pop-up)**: Khi phát hiện `Text Binary Mask` thay đổi vượt ngưỡng so với frame trước, hệ thống đợi xác nhận frame kế tiếp đã ổn định nét chữ (tránh OCR vào đúng khoảnh khắc chữ đang mờ `opacity 30%` hoặc đang phóng to dở dang), đồng thời vẫn ghi nhận mốc `start_time` tại frame bắt đầu xuất hiện.

- **Lớp 3 - Chỉ gọi AI OCR khi vùng chữ thực sự đổi**: Nếu `IoU / SSIM` của Text Mask giữa frame hiện tại và frame vừa OCR đạt mức tương đồng cao, hệ thống **bỏ qua không gọi AI OCR** mà chỉ kéo dài `end_time` của câu sub hiện tại. Nhờ đó tốc độ tăng gấp **5–10 lần**.

3. **Bộ Engine OCR Đa ngôn ngữ (Hybrid Local + Cloud)**:
   - **Chế độ CJK & Tiếng Anh (`RapidOCR` / ONNX PP-OCRv4)**: Dùng `rapidocr_onnxruntime` để phát hiện toạ độ dòng chữ (Text Detection) và nhận diện cực chuẩn chữ Trung, Anh, Nhật, Hàn mà không cần cài đặt bộ `PaddlePaddle` cồng kềnh.
   - **Chế độ Tiếng Việt (`RapidOCR Detector` + `VietOCR Recognizer`)**: Sử dụng bộ Detector của RapidOCR để cắt từng dòng chữ trong ROI, sau đó đưa qua mô hình `VietOCR` (Transformer/VGGSeq2Seq huấn luyện chuyên biệt cho dấu Tiếng Việt) để đạt độ chính xác dấu câu cao nhất.
   - **Chế độ Cloud (`Gemini Vision API`)**: Cho phép gửi batch các ảnh crop ROI đã lọc trùng lên Gemini (`gemini-2.5-flash`) để vừa OCR siêu chuẩn mọi phông chữ nghệ thuật, vừa tự động sửa lỗi ngữ cảnh.

4. **Gom nhóm & Khử trùng lặp bằng Fuzzy Matching (`RapidFuzz`)**:
   - Đôi khi cảnh nền lóa sáng làm OCR nhận diện frame 1 là "*Xin chào các bạn*" và frame 10 là "*Xin chào các bạr*".
   - Bộ **Deduplicator** sử dụng khoảng cách chỉnh sửa chuẩn hóa (`rapidfuzz.fuzz.ratio >= 85%`) kết hợp khoảng cách thời gian (`gap <= 0.35s`) để tự động gộp các đoạn liền kề có nội dung tương đương, và chọn bản văn bản có điểm tự tin (`confidence score`) cao nhất kèm ảnh crop rõ nét nhất lưu vào dự án.

3. Cấu trúc Thư mục Dự án (Project Directory Structure)

```
text
d:\Workspace\Sub magic\
├── backend/
│   ├── app/
│   │   ├── main.py                # FastAPI entrypoint, CORS, static mount (ảnh
crop/video)
│   │   ├── config.py              # Cấu hình đường dẫn cache, model, thiết bị (CUDA/CPU)
│   │   ├── database.py            # SQLite persistence (lưu Project, Subtitles, Job
state)
│   │   ├── schemas.py             # Pydantic models cho API & WebSocket events
│   │   ├── api/
│   │   │   ├── projects.py        # Quản lý tạo project, nạp video, lưu/sửa phụ đề
│   │   │   ├── jobs.py            # Khởi chạy/hủy job STT & OCR, WebSocket streaming
│   │   │   ├── models_api.py      # Kiểm tra phần cứng GPU/CPU, trạng thái tải model
│   │   │   └── export.py          # Xuất file .SRT, .VTT, .TXT (Đơn ngữ / Song ngữ)
│   │   ├── services/
│   │   │   ├── hardware.py        # Tự động phát hiện NVIDIA CUDA / ONNX Providers / CPU
│   │   │   ├── ffmpeg_service.py  # Tách audio, lấy metadata video, trích xuất frame
nhanh
│   │   │   ├── stt_pipeline.py    # Faster-Whisper + Silero VAD + Smart Word Segmenter
│   │   │   ├── ocr_pipeline.py    # Adaptive Sampler + 3-Layer OpenCV Filter + RapidFuzz
│   │   │   ├── ocr_engines.py     # Adapter cho RapidOCR, VietOCR, và Gemini Vision
│   │   │   └── llm_service.py     # Gemini API: Sửa lỗi chính tả OCR & Dịch phụ đề song
ngữ
│   │   └── utils/
│   │       └── subtitle_formatter.py # Bộ chuyển đổi chuẩn xác sang SRT, VTT, TXT
│   ├── data/                      # Thư mục lưu SQLite DB, ảnh crop ROI đối chiếu, model
cache
│   └── requirements.txt
├── frontend/
│   ├── src/
│   │   ├── components/
│   │   │   ├── VideoWorkspace.tsx  # Video Player + Canvas kéo thả ROI & Exclusion Mask
│   │   │   ├── SubtitleEditor.tsx  # Bảng phụ đề tương tác + Ảnh crop OCR + Gộp/Tách dòng
│   │   │   ├── ControlPanel.tsx    # Cấu hình chế độ STT / Hardsub OCR, chọn ngôn ngữ & AI
│   │   │   ├── ProgressBar.tsx     # Thanh tiến độ Realtime WebSocket + Log trực tiếp
│   │   │   └── ExportModal.tsx     # Tùy chọn xuất .SRT / .VTT / .TXT & Dịch song ngữ
│   │   ├── services/
│   │   │   └── api.ts              # Giao tiếp REST API & WebSocket client
│   │   ├── types/
│   │   │   └── subtitle.ts         # Định nghĩa kiểu dữ liệu Project, SubtitleSegment, ROI
│   │   └── App.tsx
│   ├── package.json
```

```
│   └── vite.config.ts
└── start.bat                        # Script khởi động nhanh cả Backend & Frontend trên
Windows
```

4. Lộ trình Triển khai (Implementation Plan)

Giai đoạn 1: Khởi tạo Hạ tầng Backend, CSDL & Nhận diện Phần cứng

- Thiết lập môi trường Python (`backend/`) và cấu trúc cơ sở dữ liệu SQLite (`projects` , `subtitle_segments`) để hỗ trợ **Auto-save** tự động mọi tiến trình.
- Xây dựng `hardware.py` và `ffmpeg_service.py` để kiểm tra GPU CUDA / CPU INT8, đọc thông số video (FPS, độ phân giải, thời lượng) và trích xuất âm thanh.

Giai đoạn 2: Xây dựng Luồng 1 — Speech-to-Text (STT Pipeline)

- Tích hợp `faster-whisper` với chế độ tự động chọn `float16` (CUDA) hoặc `int8` (CPU).
- Cài đặt thuật toán **Smart Subtitle Segmenter** dựa trên `word_timestamps` và `Silero VAD` , đẩy từng câu phụ đề vừa nhận diện qua **WebSocket** về giao diện theo thời gian thực.

Giai đoạn 3: Xây dựng Luồng 2 — Hardsub OCR Pipeline & 3-Layer Visual Filter

- Xây dựng bộ quét khung hình `ocr_pipeline.py` với vùng chọn `ROI (x, y, w, h)` và danh sách vùng che `Exclusion Masks` .
- Cài đặt bộ lọc ảnh 3 lớp bằng OpenCV (tách mặt nạ nét chữ độ tương phản cao, kiểm tra độ ổn định tránh hiệu ứng fade-in, so sánh sự thay đổi vùng chữ).
- Tích hợp `RapidOCR` (ONNX), `VietOCR` (cho Tiếng Việt) và tùy chọn `Gemini API` , kèm bộ gộp dòng trùng lặp `RapidFuzz` và lưu lại ảnh cắt (`crop_image_path`) của từng câu sub để người dùng đối chiếu.

Giai đoạn 4: Xây dựng Giao diện (Frontend) & Trình biên tập Phụ đề Trực quan

- Khởi tạo ứng dụng React + TypeScript + Vite + TailwindCSS.
- Xây dựng **Video Player kết hợp Canvas ROI Overlay**: hỗ trợ chọn Preset nhanh (`16:9` , `9:16`), kéo thả vùng ROI xanh và vẽ vùng loại trừ Watermark đỏ.
- Xây dựng **Interactive Subtitle Editor**:
    - Đồng bộ thời gian thực với video đang phát (highlight dòng sub hiện tại, click vào dòng để tua video).

    - Hiển thị **ảnh crop gốc từ frame video** ngay bên cạnh ô nhập chữ để sửa lỗi OCR cực nhanh.
    - Công cụ gộp 2 dòng liên tiếp (`Merge`), tách dòng (`Split`), chèn/xóa dòng và dịch tự động/sửa lỗi chính tả hàng loạt qua AI.

Giai đoạn 5: Bộ Xuất File Đa Định dạng & Kiểm thử Thực tế

- Hoàn thiện bộ xuất file `.SRT`, `.VTT`, `.TXT` (tùy chọn có/không kèm timestamp, xuất đơn ngữ hoặc song ngữ 2 dòng).
- Tạo script khởi chạy nhanh và kiểm thử toàn diện cả 2 luồng trên máy.